Software Engineering

CSC648/848

Henry
VILLAR

SW ARCHITECTURES AND DESIGN

Software, Multimedia and Search Architectures

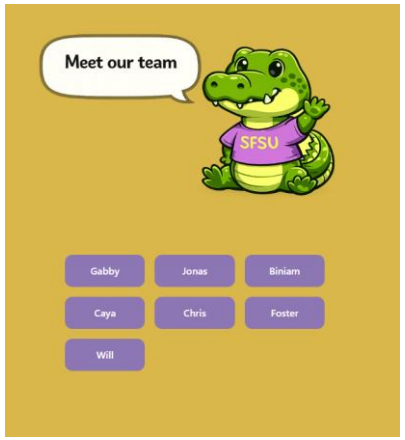
M0
Grades and any questions – Team leads Please let me know during the team collaboration.

Team 2

About our team

Team members

- Omeid Nadery
- Mikias Berhane
- Cielina Lubrino
- Laura Wong
- Brandon Sanchez
- Damian Perez
- Jason Le



This site can't be reached

34.53.13.204 took too long to respond.

Try:

- Checking the connection
- Checking the proxy and the firewall
- Running Windows Network Diagnostics

ERR_CONNECTION_TIMED_OUT

Reload

Software Engineering Class SFSU

Spring 2026

Section 02

Team 5

Team Members:

- Ian Kligman
- Adrian Aquino
- Mahvesh Wahid
- Reman Kooner
- Miguel Anguiano Hernández
- Yuejun Jiang

Team 03 Backend

Backend infrastructure and server-side development

Team Members

- Carlos Michael Rodriguez-Malak
- Cj Cancino
- Yanitsy Velazquez
- Richie Chen
- Franco Barra Avila
- Kaylen Teves
- Martin Romero

Backend server is running successfully.

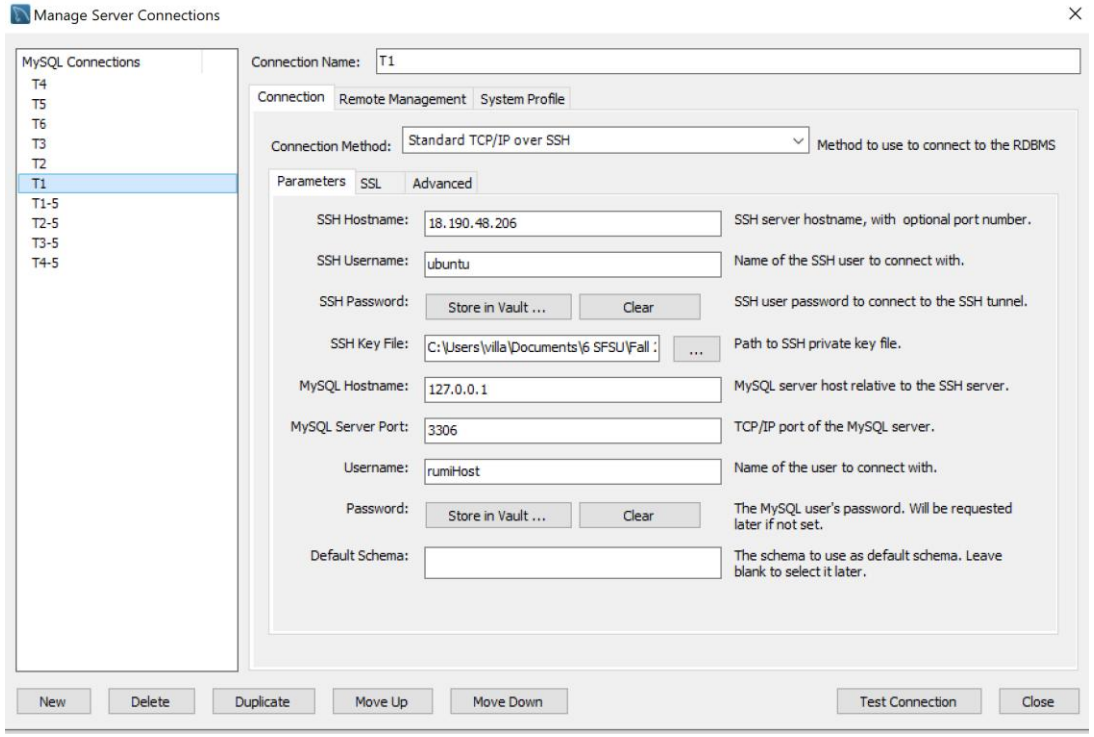
Software Engineering Class SFSU
Spring 2026
Section 2
Group 4

Welcome to Group 4's Portfolio for CSC 648

- Emma Wright
- Houdini Jezewski
- Jacob Robles
- John Torres
- Katherine Chien
- Saeed Qazi
- Sargam Bhatta Lama

M0 Feedback Cont.

DB access via MySQL Workbench



GitHub Repository

Did you…?

1. Clone your team's private GitHub repository into your individual shell account or your local computer?
2. Make sure your individual test environment is identical to the one on the server?
3. Follow the instructions? Re: Add your teams application URL or IP to the repository description? Credentials Folder? GitHub member/user?

How you will use the Milestones folder? And the Application folder?

M0 Feedback Cont.

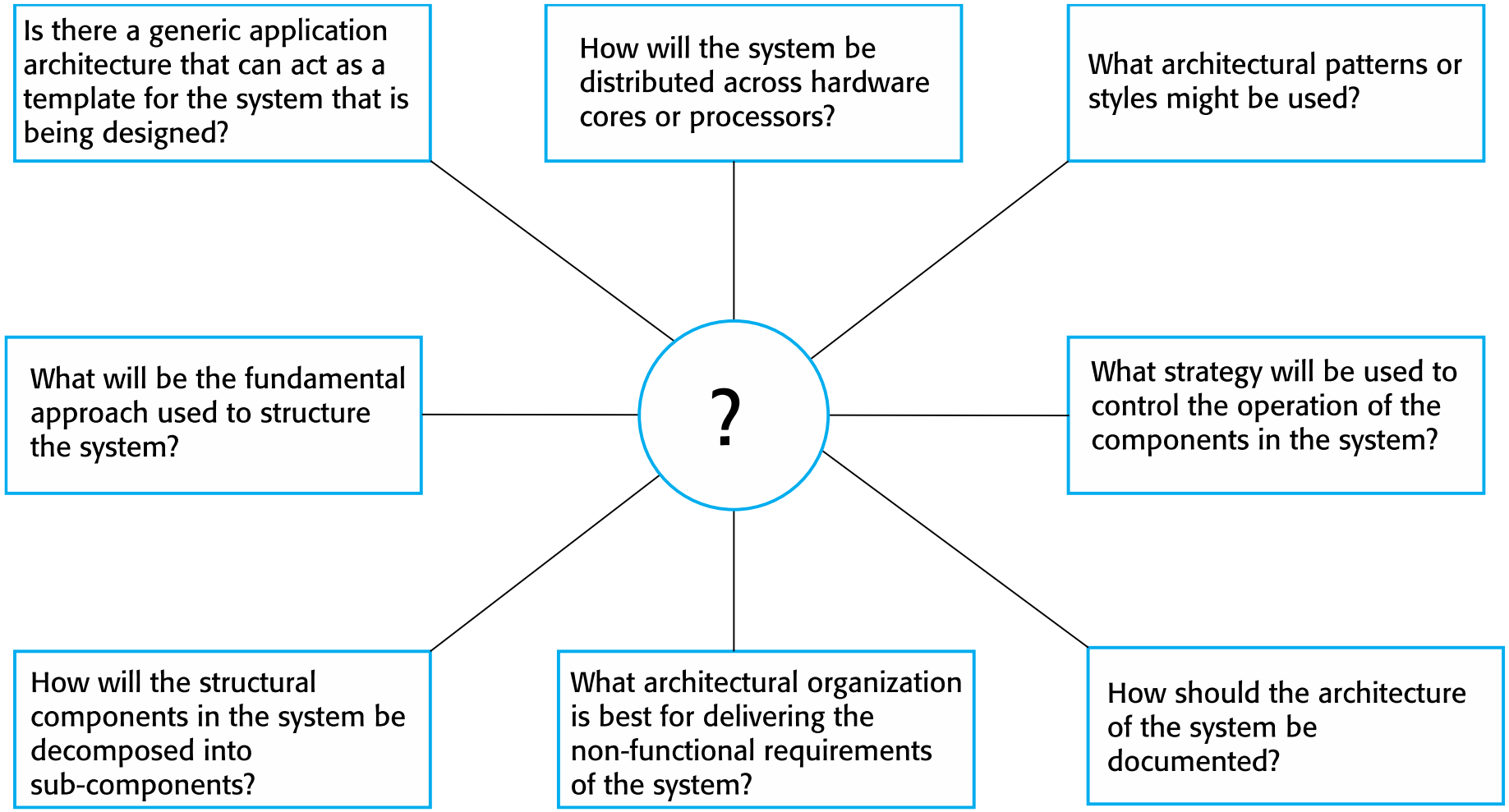
M0 Grading = 10 points

- Correctly Installed and Configured Software Stack…2 pts.
- Correctly Populated Credentials Folder in GitHub…3 pts.
- Correct team WWW page functionality, deployment and proper usage of team's Software Stack for creating web page…3 pts.
- Individual SE Dev Environment video uploaded in Canvas…2 pts.

Your Grade

- Correctly Installed and Configured Software Stack
- Correctly Populated Credentials Folder in GitHub
- Correct use of GitHub
- Correct team WWW page functionality
- Video of individual SE Dev Environment

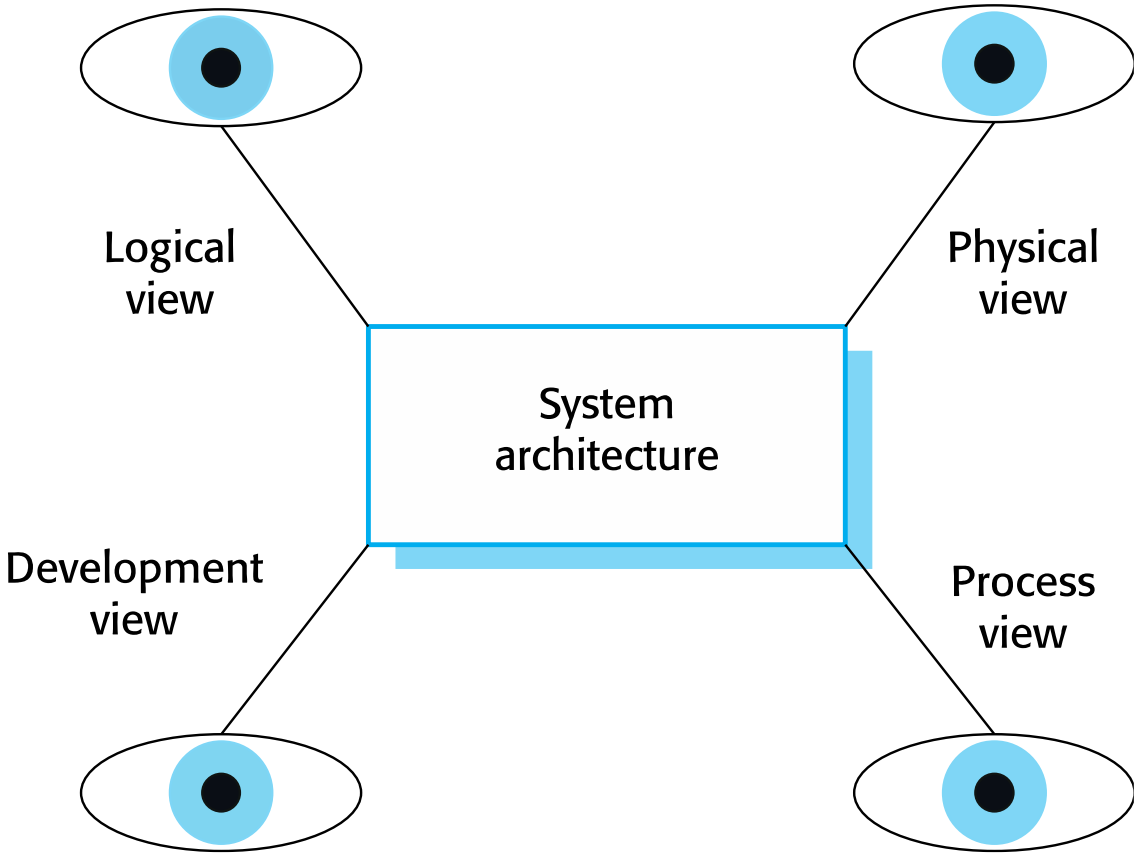
Architectural design decisions



Architecture and system characteristics

- Performance
  - Localize critical operations and minimize communications. Use large rather than fine-grain components.
- Security
  - Use a layered architecture with critical assets in the inner layers.
- Safety
  - Localize safety-critical features in a small number of sub-systems.
- Availability
  - Include redundant components and mechanisms for fault tolerance.
- Maintainability
  - Use fine-grain, replaceable components.

Architectural views



4 + 1 view model of software architecture

- A **logical view,** which shows the key abstractions in the system as objects or object classes.
- A **process view**, which shows how, at run-time, the system is composed of interacting processes.
- A **development view**, which shows how the software is decomposed for development.
- A **physical view**, which shows the system hardware and how software components are distributed across the processors in the system.
- Related using use cases or scenarios (+1)

Important

How to communicate/describe high level architecture

- Text (list components)
- Simple block diagrams (informal) – sometimes with special tools
- Unified Modeling Language – UML – formal way
  - http://edn.embarcadero.com/article/31863
  - UML Tools: https://en.wikipedia.org/wiki/List_of_Unified_Modeling_Language_tools

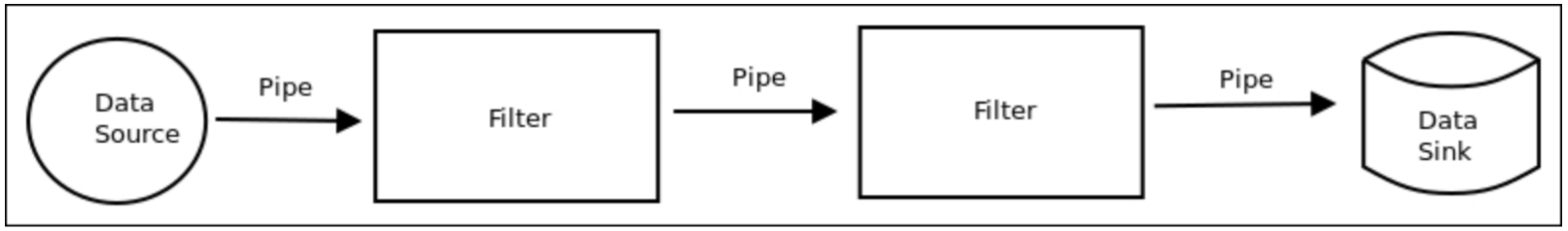
Important

Architectural design patterns

- Predefined and proven *high level* solutions for particular class of problems
- Usage: *Analyze* your application requirements then try to *map* into one of the known architectural patterns, then evaluate to chose the best
- Examples:
  - Model-View-Controller;
  - Multitier architectures. Layered Architecture (or Three (Four) Tier Arch.);
  - Repository;
  - Client-Server
- http://en.wikipedia.org/wiki/Architectural_pattern

Pipe and filter architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell - **inspired by the Unix technique of connecting the output of an application to the input of another via pipes on the shell.**).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.



Pipe and Filter Architecture ...

The pipe and filter pattern

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.13 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

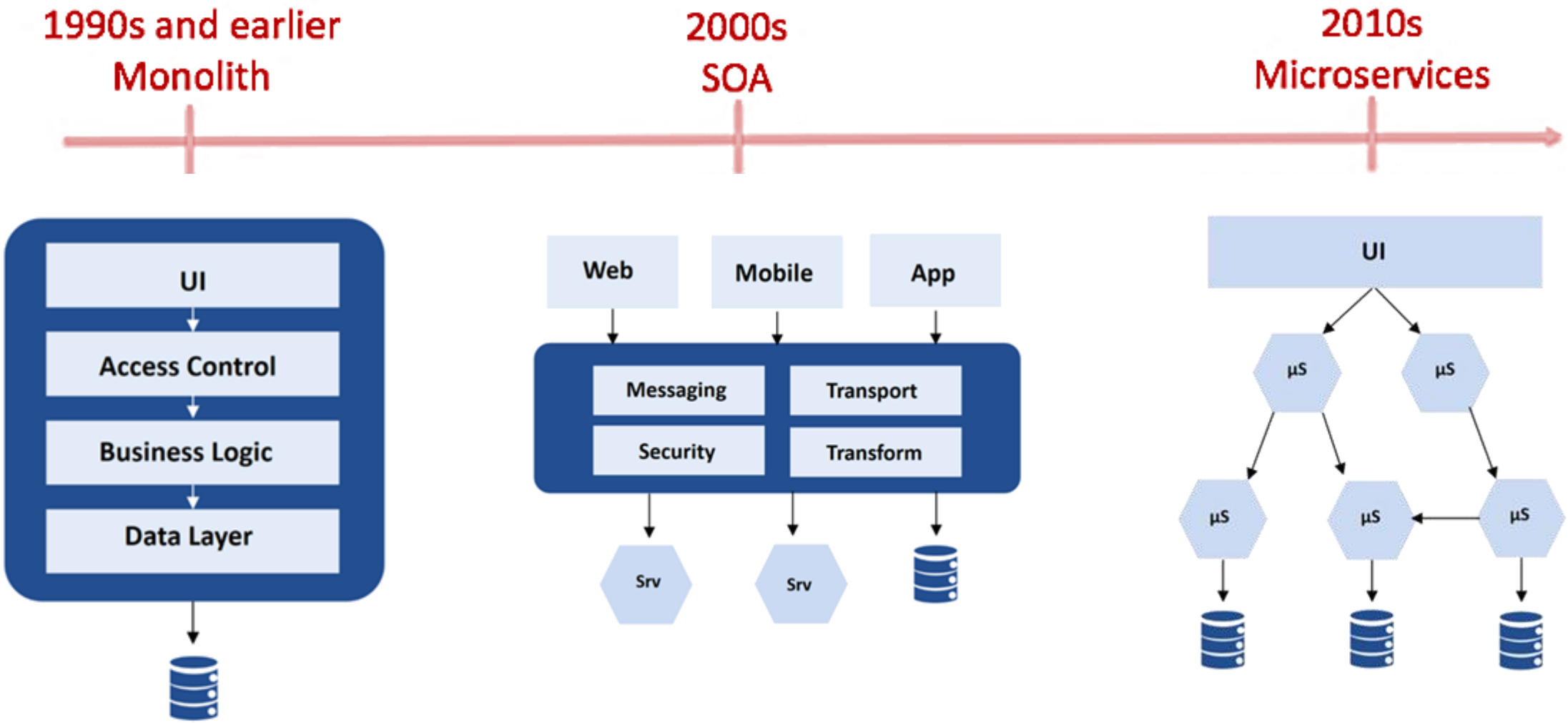
Class Activity: Canvas/Architect Role

- Mockup of an iWatch UI
- Architecture of iWatch (All Components)

Class Activity

- Architect / Designer Scenario
- Scenario: **Apple Watch Application for an AI tool interacting with any existing Health application**
  - Front End Design (UI)
  - Architecture Design Pattern
- Duration: 5 min
- Deliverables (via text, block diagrams, or UML):
  - Apple watch Mock-up of the AI tool UI
  - Chosen Architecture Design Pattern

Legacy Apps vs Modern Apps



Legacy Apps vs Modern Apps – Seriously talking *(human-oriented)*

1990s and earlier
Monolith

- Large monolithic code base
- Scaling can be difficult
- Continuous deployment is difficult

2000s
SOA

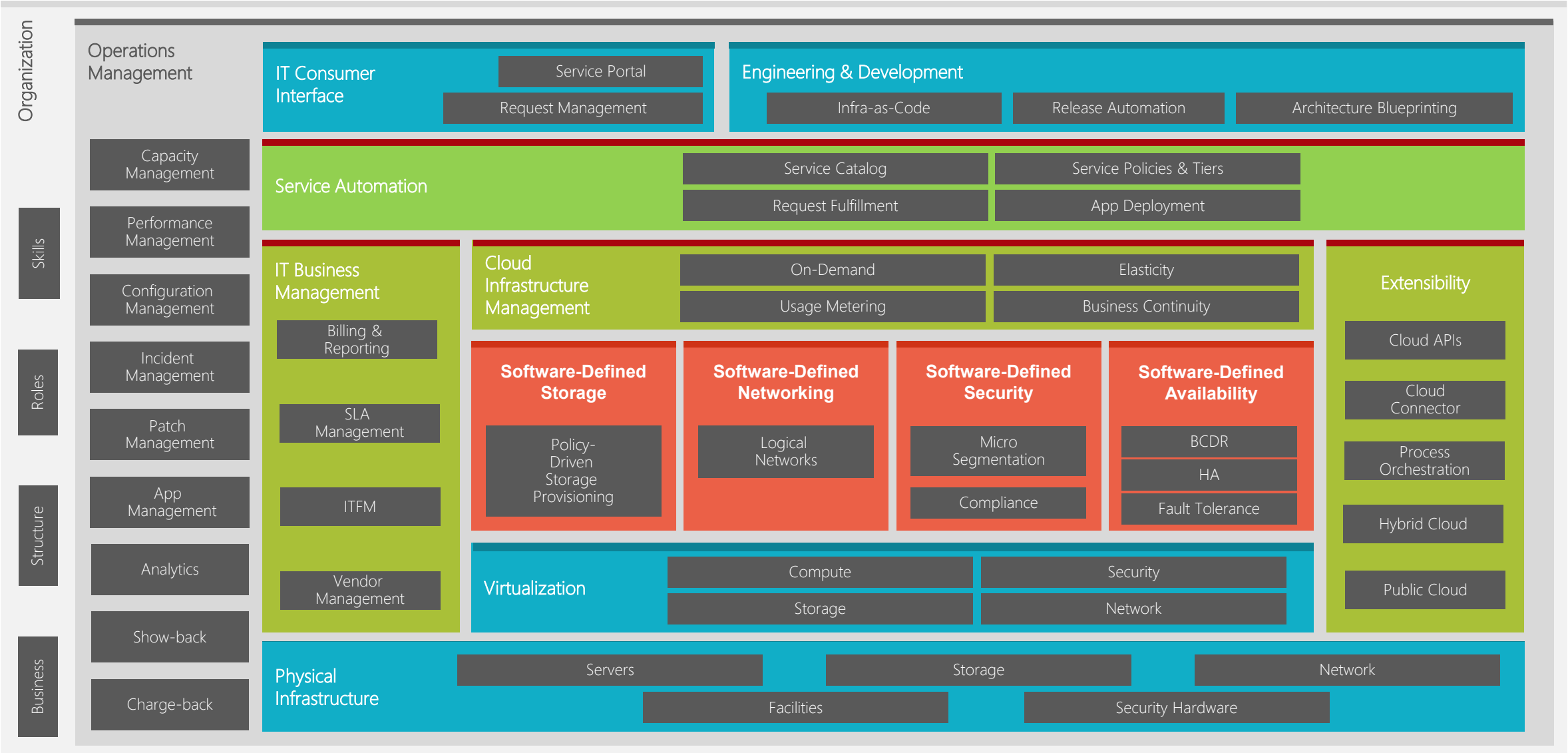
- More lines of code
- Greater emphasis on re-usability
- Stateful
- Enterprise service bus
- Traditional databases

2010s
Microservices

- May have < 100 lines of code
- Focus on decoupling
- Stateless
- Simple messaging system
- NoSQL databases

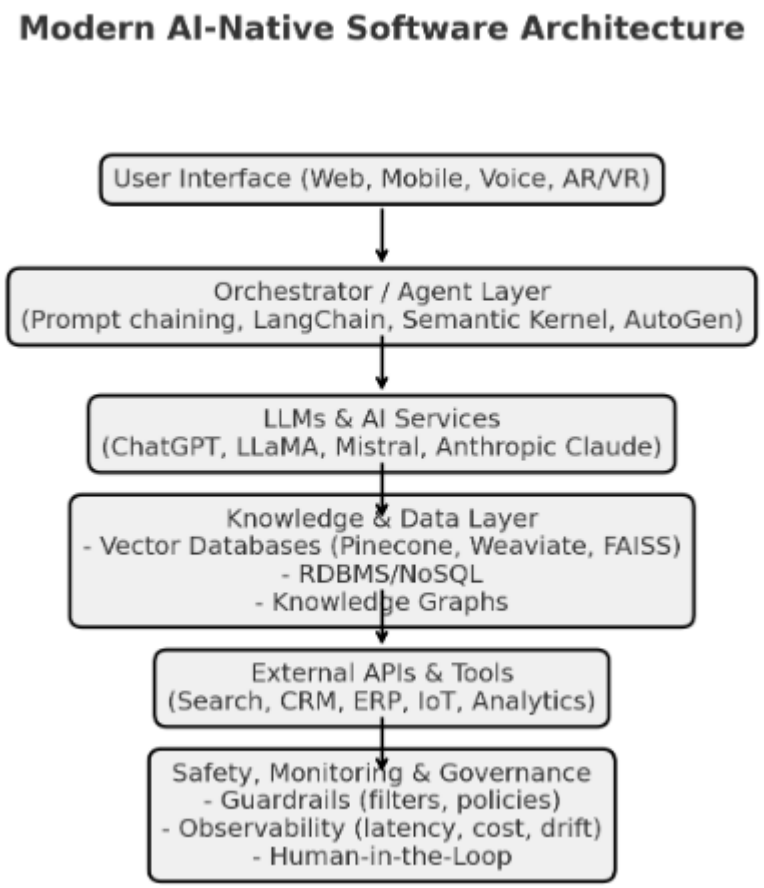
Private Cloud Reference Architecture

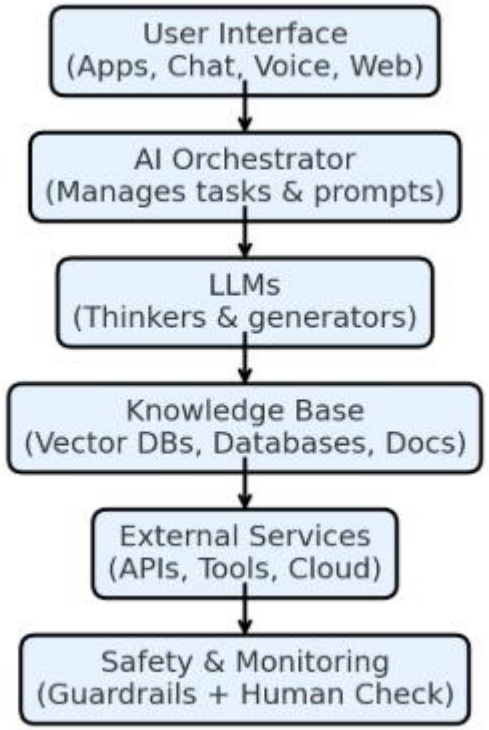
VMware



AI Architecture

AI-native software stack





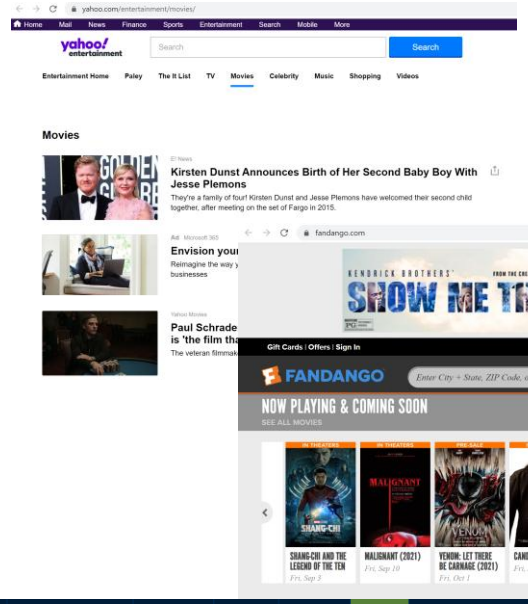
Multimedia Architectures and **Search** Design Patterns

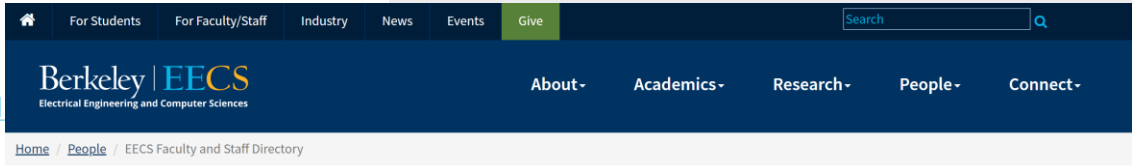
Multimedia Information Systems

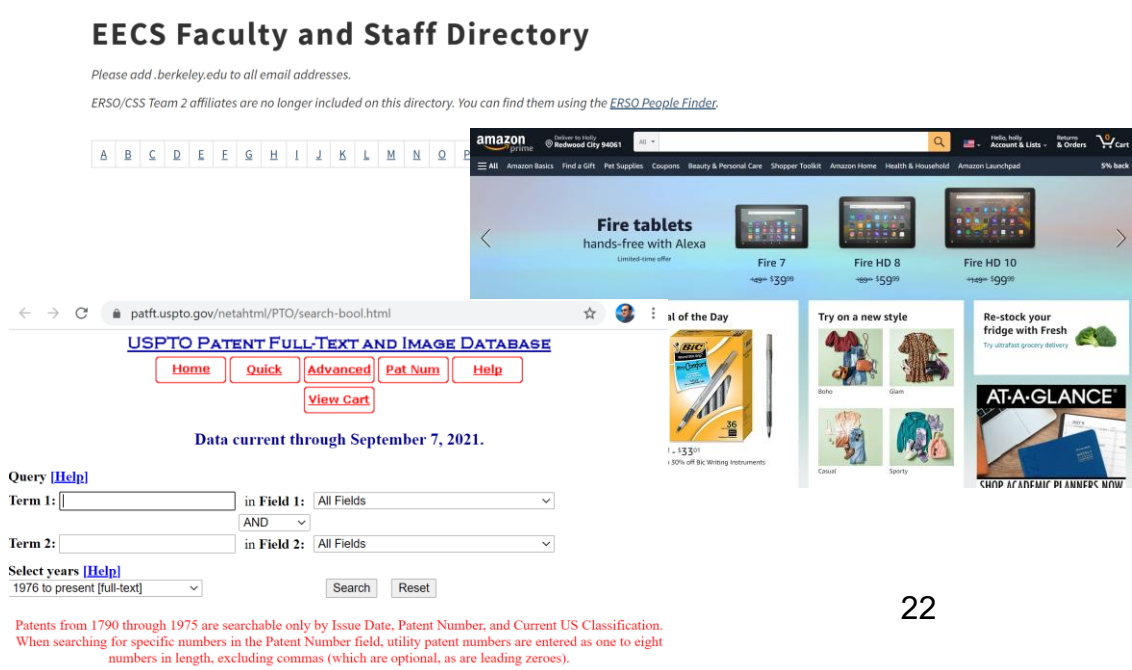
- Information systems containing, among other types of data, multimedia data
- General uses:
  1. *Find*: *Search and browse* to find the right information
  2. *View* the information to make a decision
  3. *Use* the info (play, download, burn CD, edit, buy…)
- Requires proper data capture, indexing, storage, delivery and rendering
- Examples: DB of music, videos, training material, e-commerce, marketing, maps

Search examples

- Movie search
  - https://www.yahoo.com/entertainment/movies/ (not a good one – WHY?)
  - http://www.fandango.com/ (better)
- Alphabetical/list/directory
  - https://www2.eecs.berkeley.edu/Directories/directory-nostudents.html?_ga=1.82227989.20921
- Product – search with categories and filters
  - www.amazon.com
- Media (stock images)
  - http://www.dreamstime.com/?gclid=CKqViLDJyKsCFRAaQgod2Ba21w
- Parametric/boolean (patent search)
  - http://patft.uspto.gov/netahtml/PTO/search-bool.html

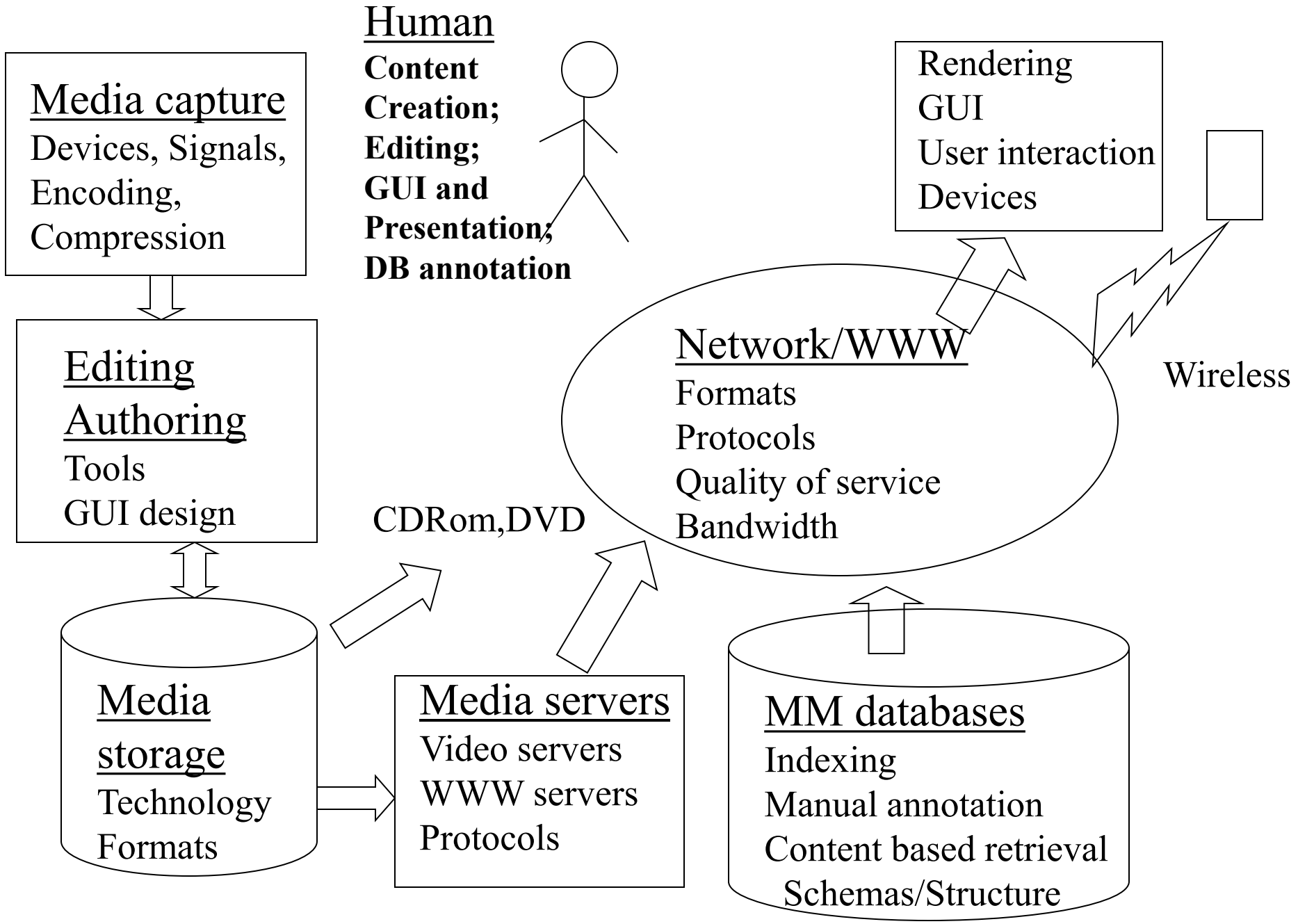






Multimedia "lifecycle"

Important



MM “programming”

- Content
  - Creation
  - Capture
  - Editing
  - Encoding
- Data organization: Metadata and annotations for DB (data about MM like author, synopsis, title etc.), schemas, DB management
- Presentation, GUI development

Each is very human intensive and involves different skills

Important

Multimedia Data Elements

- *Raw Media*: bits containing content that is played or viewed
  - Color images
  - Graphics
  - Animation
  - Audio
  - Video
- *Supporting data*
  - Used for search (*metadata*)
  - Supporting info (not used for search): Text and supporting information (text, HTML, PDF, PowerPoint, keyframes, thumbnails etc.) to display items (low and high detail)

Important

MM Inf. Systems >> Functionality >> Search

simple

- *Browse by category* (e.g. from the list)
- *Free Text entry field* (possibly with domain selection pulldown)
- *Parametric search e.g.* form-like search (can put value limits on variables like title, price, image size, video duration) – fixed structure
- *Advanced, Boolean* (can create ad-hoc logical combinations of search conditions)
- *Content based retrieval* – search based on automatically extracted information from raw media (give me images "like" this) - R&D topics for now

complex

Components of search functions

Important

- *Metadata* – describe items in terms of text or numerical data which is used for searching. Input to search or indexing algorithm

- *Index* - a data structure enabling search, computed off-line during indexing phase. Input: search arguments; Output: pointers to relevant items. Issue: when you add the data do you need to reindex the whole DB or not?

- *Search arguments* (what is used to search with) *and* ***scope*** (what domain is searched e.g. the whole WWW, local WWW etc.)

- *Search algorithm* – steps and functions used to do indexing (often called indexing alg.) or search. KW search, text search, SQL, NL etc. Can be done on index (which is pre-computed) or run-time (e.g. running SQL)

- *Search UI*: how is it offered to the user: a) entering search arguments and scope; b) viewing results; c) refining

STUDY MATERIALS

Search systems performance measures

Important

- *Precision*: out of R retrieved elements, how many (RR) are relevant: RR/R
- *Recall*: out of total of NR relevant items in the whole database/domain, how many are retrieved (RR) in specific query: RR/NR
- (Precision is usually easiest to measure in WWW context since NR (total number of relevant items) is usually not known)
- *Indexing efficiency*: how long it takes to index the database
- *Run time efficiency*: how long does it take to perform search

Important

SW Frameworks

- Platforms to help develop SW applications – an “environment” including tools, APIs, compilers etc.
- http://en.wikipedia.org/wiki/Software_framework
- Offer tools and environment on top of APIs
- Provide easier ways to build SW systems with the set of prebuilt major functions (tables, login, calendar etc.) and ensures for example cross browser and mobile device compatibility (Bootstrap)
- Commercial and open source solutions available

BUT

Important

Questions to ask re: Frameworks

- Functionality: will it do what I need to do now AND in the future
- Maintainability: how do I maintain the system built by this framework
- Licensing, terms of use, price
- Cross platform issues
- Learning curve of the team to use it
- Availability of support and documentation
- Market share, how widely used, stability of the company producing it, availability of skilled people

Do not believe the marketing – VERIFY: install, try and test it for YOUR application and functions

If you decide to use specific frameworks read documentation and follow instructions (this is recurring problem in our SE class)

Use frameworks properly and do not hack around it!

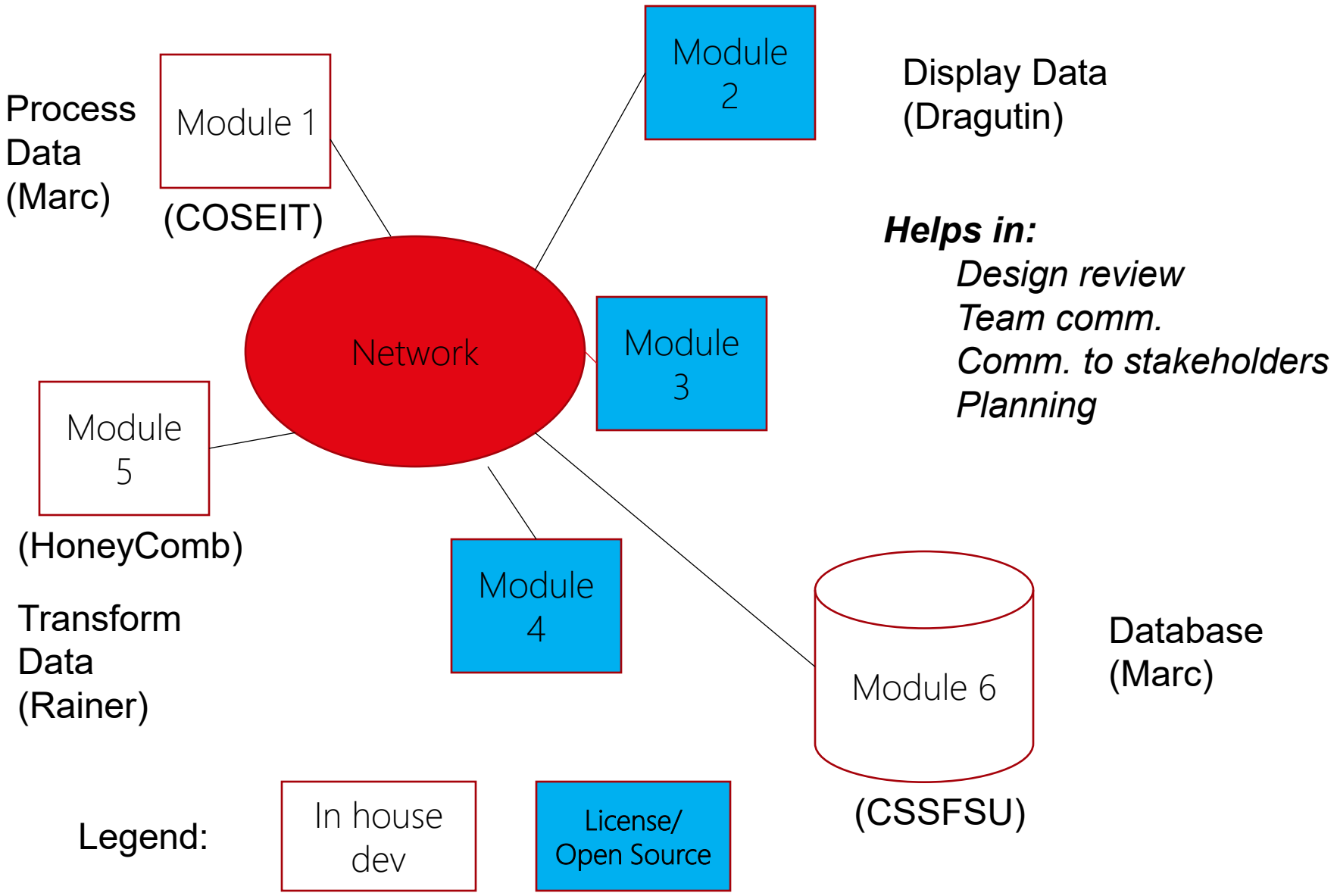
Important

Architectural design

"Process of designing SW system organization"

- Driven by non-functional specs (which are usually driven by business needs and cost)
- Facilitates discussion about the SW design (needs to be *reviewed and adopted*)
- Needs to be followed and enforced (or revised)
- Helps in planning and cost estimates
- Helps communicate the design to the team and other stakeholders
- Always do high-level architecture review early in the process

High level arch: show major systems, function, owners, location



Important

Questions to ask in Arch. Design process (1)

- Can the application be mapped to known architectural solution or design pattern
- How to distribute the system across multiple processors
- Does the architecture support the system, data size and load requirements
- How to decompose system into subsystems – MODULAR design (high intermodule cohesion, loose coupling)
- Does the architecture satisfies non-functional requirements and costs
- How it should be reviewed and documented
- What existing farmeowrk can be used best to implement it

Questions to ask in Arch. Design process re implementation (2)

Important

- Are the chosen tools and frameworks reliable, used in the marketplace, tested and supported
- Does the team know how to use them
- Licensing, legalese, cost
- Maintenance
- Stability of the provider

How to decompose system into subsystems – system design

Important

- **This is crucial to good SW design irrespective of technologies used!**

- Partition and group requirements and define subsystems
- Specify subsystems – make it *modular*:
  - *Functionality* (well grouped and cohesive, focus on one category of functions)
  - *Interfaces* (ensure lose coupling, go for standard ones, well accepted in the market)

Important

Goal of partitioning – modular design

- ***Modularity:***
  - *Intra-subsystem cohesion*: The subsystem has well defined and cohesive function (I.e. data gathering, not data gathering and data processing).
    - Test: can you describe its function in one simple sentence
  - *High intersystem independence or loose coupling* (I.e. separate data storage from data rendering)
    - Clean, well defined preferably standard interfaces
- Benefits:
  - Maintainability
  - Easy to debug, QA
  - Easy to optimize for security, speed

- "With modular programming, concerns are separated such that modules perform logically discrete functions, interacting through well-defined interfaces." ref http://en.wikipedia.org/wiki/Modular_programming

- Examples" MVC, SQL etc.

Examples of partitioning

- System/method X has function to obtain *and* process data
  - Wrong! Make two separate subsystems/methods/classes. Why?
- Group all data access into one subsystem, one code. Test it well and enforce that only this code is used.
  - Why? Think of testing and tuning for performance or security
- Use industry standard APIs (application programming interface) and communication protocols
- Better use suboptimal but standard API and protocols than custom one of your own

Architecture design patterns: structure

- Description (not the code)
- Example
- When used
- Advantages
- Disadvantages

Example of metadata for movie

- Video format
- Duration
- Title
- Rating
- Time taken
- Author
- Actors
- Director
- Credits
- Brief description
- Number of scenes
- Promotional images
- Movie trailer (type, format, duration)
- Music scores (number,type,format)
- Links to press releases
- Number of scenes

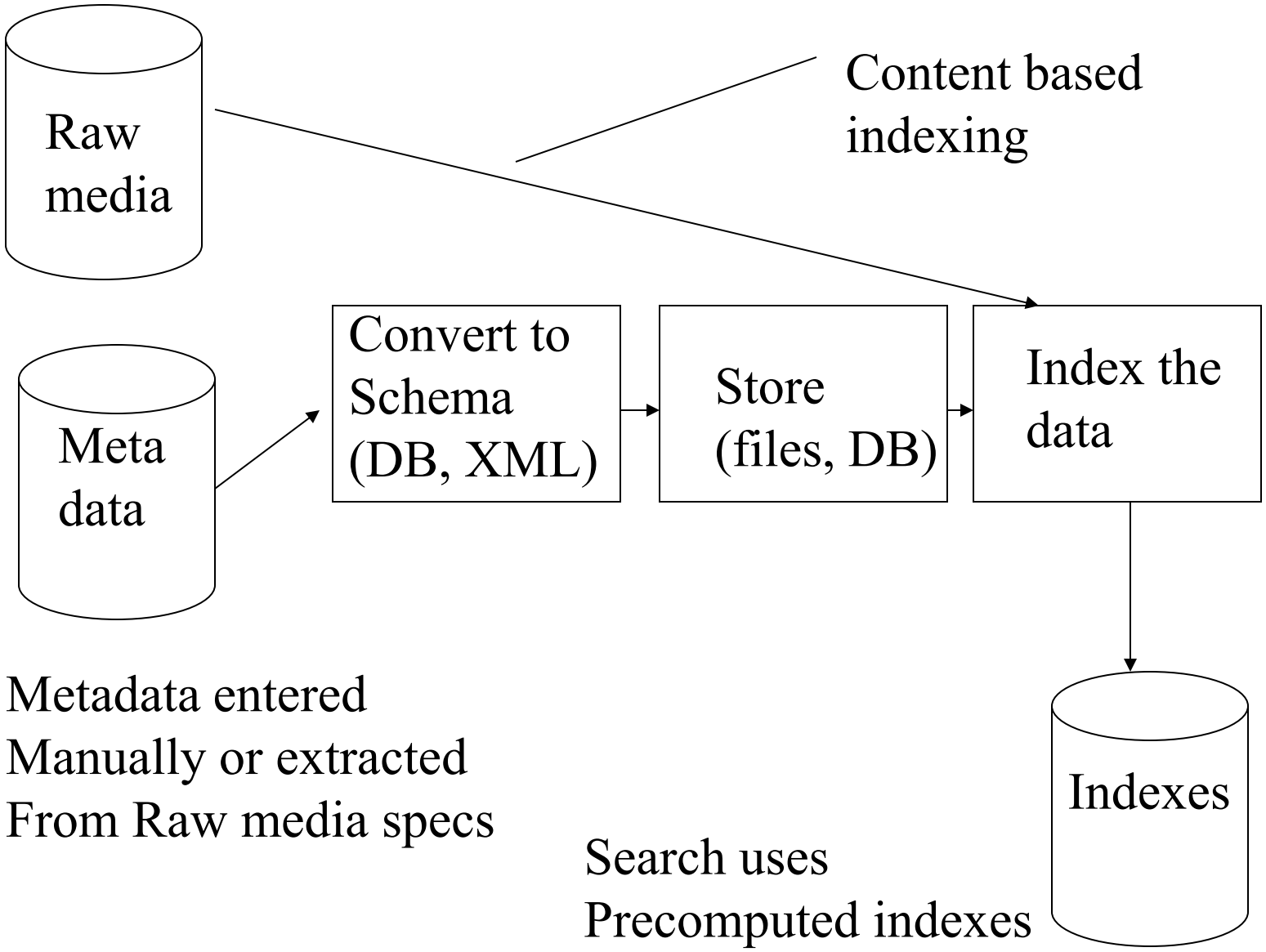
- For each scene
  - Description
  - Actors
  - Duration
  - Keyframes
  - Video

Metadata encoded in XML or DB tables

How to design metadata for media search

- This is more like a librarian and information organization effort and NOT a coding job
- Think of:
  - What users want to search for
  - Characteristics of the media content (movies, images, music) commonly used to search for it
- Avoid too many search fields which are ANDed – most often produces empty result
- Pull-down preferred from free text typing – avoids typos and also refes to existing content

Indexing for MM search



Metadata entered
Manually or extracted
From Raw media specs

Search uses
Precomputed indexes

MM Inf. Systems >> Architecture design patterns

- Quick overview of basic three tier architectures, with emphasis on the WWW

- Specifics related to MM Inf. Systems

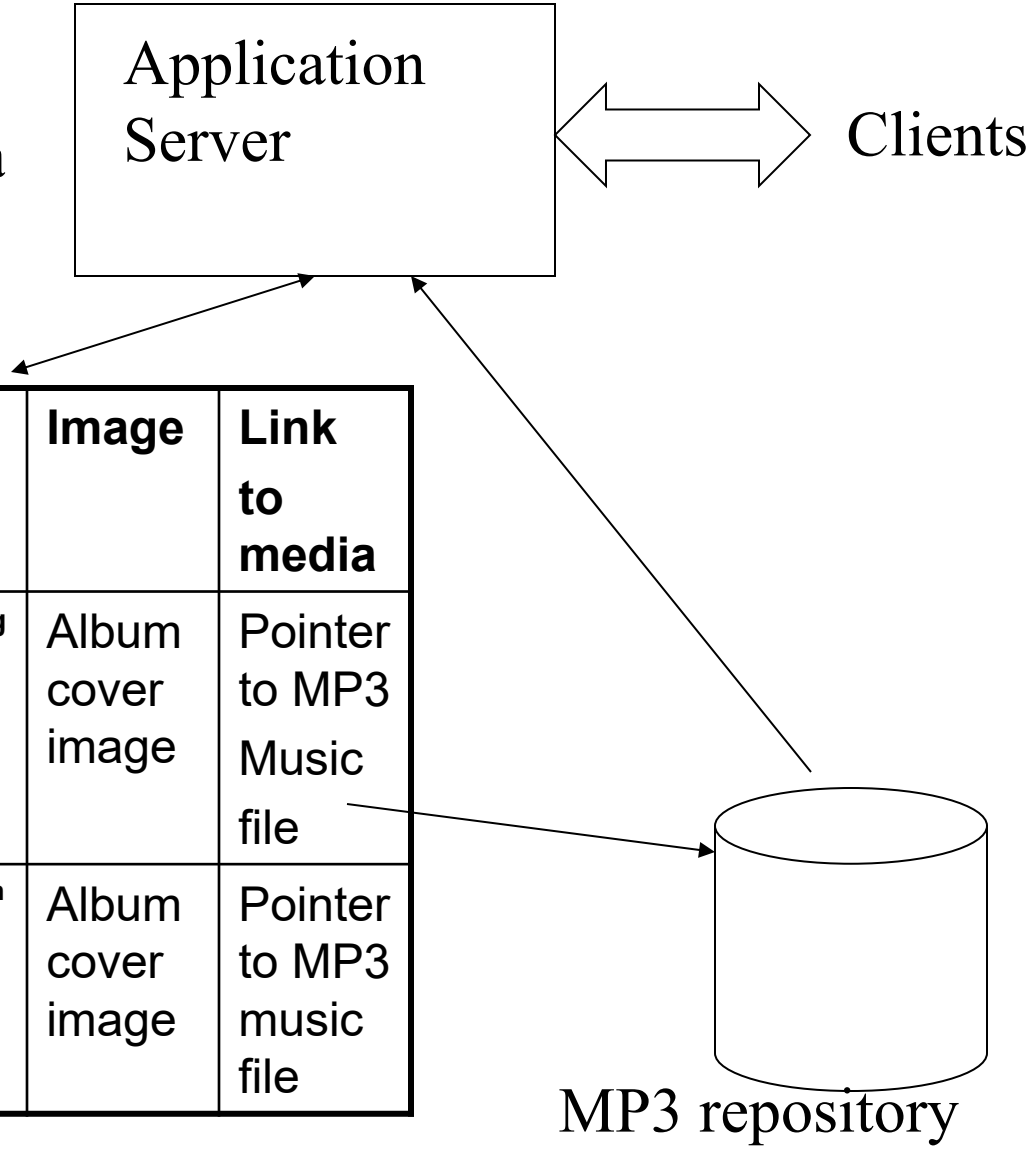
Important

MM Inf. Systems Architectures – historical evolution

- Mostly client server or layered arch. with varying degree of data integration in the back-end
  - **Media in standalone on files**, CDRom: collection of interlinked hard wired files; **Traditional DB** with metadata in DB tables and pointers to raw medias on file systems
  - **BLOBs**: Binary Large Objects – integrates media files into the DB
    - https://en.wikipedia.org/wiki/Binary_large_object
    - https://dev.mysql.com/doc/refman/5.7/en/blob.html
  - **Media content management**– full applications buiolt on top of the above
- **Trade-offs**: cost, complexity, administration, performance, security, recovery, manageability etc.

Important

MM Systems
With DB and raw media
In separate files



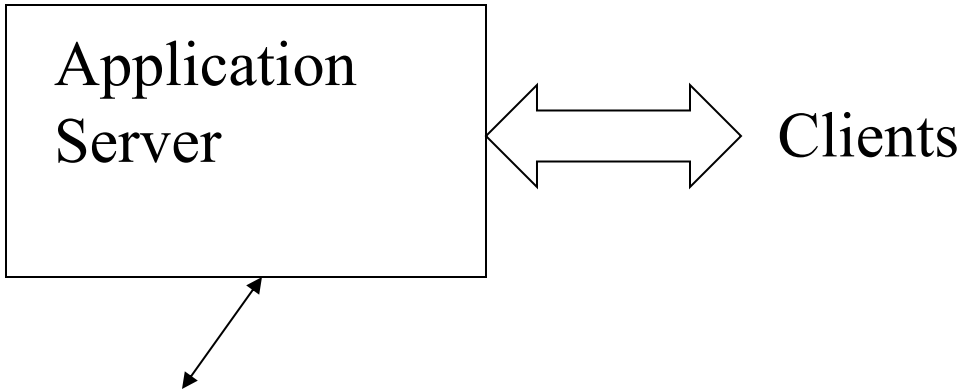
ID	Author	Title	Image	Link to media
Item1	Miles Davis	Something Else	Album cover image	Pointer to MP3 Music file
Item2	Rolling stones	Satisfaction	Album cover image	Pointer to MP3 music file

Table in relational DB

Images can be stored in DB tables or externally

Important

MM Systems
With DB and BLOBS



ID	Author	Title	Image	BLOB
Item1	Miles Davis	Something Else	Album cover image	raw MP3 Music file
Item2	Rolling stones	Satisfaction	Album cover image	Raw MP3 music file

Table in relational DB

Important

MM Inf. Systems with DB and raw media in separate file system

- Pros:
  - Simple
  - Efficient
  - Existing media data kept in place
  - Media data in their natural format
  - Easy to load and access media data –no DB overhead
- Cons:
  - No admin and access control
  - Hard to manage access by many users
  - No transaction security (backup, roll-back etc.)
  - Data on files easily gets out of synch with metadata in DB
  - No leveraging of all the DB tools and functionality such as distribution etc.
- Files vs. BLOBs is complex and controversial (efficiency, overhead, also deepens on file size)

BLOBs may Help here

Key problem in use of classic relational DB for metadata management

- Metadata vary in format and change in time
- Often have messing elements
- Hard to fix it ahead of time (which is required for Relational DB implementation)
- Too brittle and inflexible
- Slow in data ingest/input
- Hard to use in real applications
- BUT….<u>NO-SQL to the rescue</u>

Important

NEW: No-SQL for managing metadata

- Driven by Internet world (data harvesting)
- New technology – based on name-value pars -Jason objects
  - https://en.wikipedia.org/wiki/NoSQL
- Coverts data into name-value pairs in strings
- Can be stored in some databases as strings
- Fast ingest (import/input) and search
- Allows SQL access on top of no-SQL
- Not "safe" or ACID as relational DB but this is OK for metadata/internet/search
- Major improvement in for metadata management and for search engine databases – very good high level comparison of SQL and NoSQL for data analytics and search engines
  - https://www.forbes.com/sites/metabrown/2018/03/31/get-to-know-relational-and-nosql-databases-that-power-big-data-analytics/#719d56091943

How to store image paths in DB column

- Avoid absolute hard coded path names – makes your app very "brittle" and sensitive to changes in deployment directory
- Use relative paths e.g. store image name and append relative path to the root of appl. or current work folder
- Some discussion is here
  - https://www.daniweb.com/programming/databases/threads/475006/storing-image-file-paths-into-database
  - https://www.quora.com/What-is-the-best-way-to-store-100-images-in-a-MySQL-database-in-this-case
- Can also use UUID – universally unique identifier
  - https://en.wikipedia.org/wiki/Universally_unique_identifier

Important

Suggested DB organization when media item (image, video, sales item, real-estate post etc.) has associated *category used for search*

- Have a basic DB table with media ID, title, decryption etc. and CATEGORY as a foreign key
  - Media data referenced as pointer to file or BLOB
- Have separate DB table for CATEGORY
- Benefits:
  - If categories are changed or edited you change only DB CATEGORY table
  - DB CATEGORY table drives all UI menus (search, upload) – much easier to manage vs. hard coding categories in java script

Backend for search combining categories (exact) and text (substring) search

- Case study: you want to search items by combining:
  - Category (e.g. furniture, electronics…)
  - Fuzzy text search on text (e.g. concatenate item title, description in one DB field (e.g "SOFA red sofa by Lui XVI")
- One <u>simple</u> solution (OK for small data size):
  - SQL precise search for *categories* from DB column called *categories*
  - *%like* search on *text field* e.g. item description + item title
    - http://www.mysqltutorial.org/mysql-like/
  - (MySQL and other DB have more text search options)
    - https://dev.mysql.com/doc/refman/5.7/en/fulltext-search.html
- *(In many applications like product or service searching fuzzy and approximate search are desired)*

Our own tutorials for app infrastructure/architecture (documented code) – feel free to use and modify

- Tutorial with nodejs, developed by our former student and TA Nicholas Stepanov
  - https://medium.com/@nicholasstepanov/search-your-server-side-mysql-database-from-node-js-website-400cd68049fa
- Tutorial with flask, developed by our SE instructor Jose Ortiz
  - https://medium.com/@joseortizcosta/search-utility-with-flask-and-mysql-60bb8ee83dad
- Tutorial with PHP, by Jose Ortiz
  - https://medium.com/@joseortizcosta/search-utility-with-php-and-mysql-as-backend-server-technologies-d3dac5128d8
- How to use:
  - Study code to learn
  - Customize for your app; deeply and test then put on master branch
  - Document well, establish good APIs
  - Use as templates/architecture to guide each team member
  - Perform constant code reviews to ensure people follow the templates and APIs

How to organize the DB – from our CTO

- It is NOT a requirement to have your database and application on different servers. as in industry ➔ make it simple

Main options

- Main production DB and local DB for each member
  - Each member deploys and maintains their own localhost database (e.g. on their laptops) and the team has one main production DB for your application.
  - However, keeping these databases consistent is going to be a challenging task
- Main DB and Test DB on deployment server, team accesses them
  - Run 2 databases (under the same DBMS software) on the remote server: production DB and a Test db.
  - Both should be maintained by the same team member.
  - The test and production database should be a similar as possible.

Review questions

- Know definition and benefits of arch. Design patterns
- Know main types of design patterns, when to sue them, pros and cons
- Define SW modularity and its benefits
- What are APIs and what is the diff. vs. ABIs
- What are SW frameworks, their benefits and important things to ask before using them
- What are design patterns
- Draw the full lifecycle diagram of MM information system
- Typical MM data elements in MM Information system
- Ways to measure search systems performance

Review Questions

- Outline historical evolution of MM information systems
- Explain architectural differences between DB and MM in separate files, vs. integrated media DB. What are pros and cons? How is the data organized in the first case?
- You are developing MM Inf. System of medium size, with limited user base, where security of access and transaction integrity is not critical. What approach will you use from the two above?
- You are developing medical image DB with strict rules on security and access, and large user base. What approach will you use from the two above?
- What is no-SQL and what are its advantages re: metadata management

Managing people

- People are an organization's most important assets.
- The tasks of a manager are essentially people-oriented. Unless there is some understanding of people, management will be unsuccessful.
- Poor people management is an important contributor to project failure.

People management factors

- Consistency
  - Team members should all be treated in a comparable way without favourites or discrimination.
- Respect
  - Different team members have different skills and these differences should be respected.
- Inclusion
  - Involve all team members and make sure that people's views are considered.
- Honesty
  - You should always be honest about what is going well and what is going badly in a project.

Class Activity

Role-based scenario (Topic: People Management / Team dynamics)

Case study: Individual motivation

Alice is a software project manager working in a company that develops alarm systems. This company wishes to enter the growing market of assistive technology to help elderly and disabled people live independently. Alice has been asked to lead a team of 6 developers than can develop new products based around the company's alarm technology.

Alice's assistive technology project starts well. Good working relationships develop within the team and creative new ideas are developed. The team decides to develop a peer-to-peer messaging system using digital televisions linked to the alarm network for communications. However, some months into the project, Alice notices that Dorothy, a hardware design expert, starts coming into work late, the quality of her work deteriorates and, increasingly, that she does not appear to be communicating with other members of the team.
Alice talks about the problem informally with other team members to try to find out if Dorothy's personal circumstances have changed, and if this might be affecting her work. They don't know of anything, so Alice decides to talk with Dorothy to try to understand the problem.

Case study: Individual motivation

After some initial denials that there is a problem, Dorothy admits that she has lost interest in the job. She expected that she would be able to develop and use her hardware interfacing skills. However, because of the product direction that has been chosen, she has little opportunity for this. Basically, she is working as a C programmer with other team members.

Although she admits that the work is challenging, she is concerned that she is not developing her interfacing skills. She is worried that finding a job that involves hardware interfacing will be difficult after this project. Because she does not want to upset the team by revealing that she is thinking about the next project, she has decided that it is best to minimize conversation with them.

Discussion

- What do you think about Alice's actions?
- What do you think about Dorothy's actions?
- In general what do you think about the whole situation?

Comments on case study

- If you don't sort out the **problem of unacceptable work**, the other group members will become dissatisfied and feel that they are doing an unfair share of the work.
- **Personal difficulties** affect motivation because people can't concentrate on their work. They need time and support to resolve these issues, although you have to make clear that they still have a responsibility to their employer.
- Alice gives Dorothy more design autonomy and organizes training courses in software engineering that will give her **more opportunities** after her current project has finished.

SW Engineering Process Components

1. *Requirements and Specifications*
2. *Design and Implementation*
3. *Validation*
4. *Maintenance and Evolution*

SW Engineering Process Components

1. *Requirements and Specifications*
2. *Design and Implementation*
3. *Validation*
4. *Maintenance and Evolution*

Thank You